

# Prognozowanie ilościowe na rynkach finansowych jako zadanie rozpoznawania wzorców w dwuwymiarowej grafice komputerowej

Jan Hordyński<sup>1</sup> i Krzysztof Łaczmański<sup>2</sup>

<sup>1</sup> WSB Merito, Wrocław

<sup>2</sup> WSB Merito Wrocław

wrx100553@student.wroclaw.merito.pl

wrx100859@student.wroclaw.merito.pl

## Abstract.

Artykuł prezentuje autorskie podejście do ilościowego prognozowania na rynkach finansowych, w którym klasyczną analizę szeregów czasowych zastąpiono metodami widzenia komputerowego (Computer Vision). Zamiast przetwarzać surowe dane liczbowe, model analizuje historyczne zachowania cen w formie znormalizowanych obrazów (wykresów świecowych z nałożonymi średnimi kroczącymi). Do identyfikacji momentów wybicia wykorzystano sieć konwolucyjną o architekturze ResNet-18. Aby ograniczyć negatywny wpływ rynkowego szumu, wprowadzono dynamiczne etykietowanie oparte na rynkowej zmienności oraz zmodyfikowaną funkcję straty, która promuje znajdowanie najbardziej zyskowych trendów. Wyniki pokazują, że model potrafi skutecznie wyodrębnić wizualne wzorce geometryczne. Praca zawiera również analizę przydatności biznesowej zastosowanych metryk, ze szczególnym uwzględnieniem ograniczeń wskaźnika F1-Score w systemach transakcyjnych.

**Keywords:** wizja komputerowa · konwolucyjne sieci neuronowe · handel algorytmiczny · prognozowanie finansowe.

## 1 Wstęp

Prognozowanie zachowań rynków finansowych to zadanie wymagające pracy z danymi o bardzo dużej zmienności i wysokim poziomie szumu. W dzisiejszych realiach, gdzie informacje przepływają błyskawicznie, pojedynczy nieoczekiwany news czy nawet wpis w mediach społecznościowych potrafi wywołać nagłe zawirowania cen. Z tego powodu klasyczne modele statystyczne, opierające się wyłącznie na jednowymiarowych ciągach liczbowych, bywają bardzo podatne na pomyłki.

Doświadczeni inwestorzy często podchodzą do problemu inaczej – korzystają z analizy technicznej. Obserwując wykres wizualnie, potrafią odfiltrować chwilowe, chaotyczne wahania i dostrzec szerszy kontekst rynkowy, ignorując to, co jest tylko chwilowym szumem.

Niniejszy projekt czerpie z tego podejścia i przekształca numeryczny problem przewidywania cen w zadanie rozpoznawania obrazu. Generując okna z wykresami, pozwalamy głębokiej sieci neuronowej ResNet-18 działać podobnie jak ludzkie oko – wyodrębniać cechy przestrzenne i wyszukiwać formacje geometryczne zwiastujące wybicie na głównych parach walutowych (Forex).

## 2 Metodologia

Proces stworzenia systemu podzielono na kilka spójnych etapów: od zebrania surowych danych, przez ich przetworzenie na obrazy, aż po trening samej sieci.

### 2.1 Pozyskiwanie danych i generowanie obrazów

Skorzystano z 10-letnich historycznych notowań dziennych dla dziesięciu najpopularniejszych par walutowych. Dane podzielono na przesuwające się, 20-dniowe okna czasowe. Dla każdego takiego okna wygenerowano kwadratowy wykres świecowy na czystym, czarnym tle. Zrezygnowano z rysowania jakichkolwiek osi czy wartości liczbowych, zostawiając jedynie same świece oraz nałożone na nie linie 10- i 20-dniowych średnich kroczących

Bardzo ważnym krokiem była lokalna normalizacja (Min-Max). Rozciągnęła ona proporcje każdego wykresu do tego samego przedziału  $[0, 1]$ . Dzięki temu model uczy się relatywnych kształtów formacji rynkowych, a nie zapamiętuje sztywnych, bezwzględnych poziomów cen, które ciągle się zmieniają.

### 2.2 Etykietowanie dostosowane do zmienności

Kluczowym wyzwaniem w prognozowaniu finansowym jest zdefiniowanie znaczących etykiet docelowych. Wdrożono dynamiczny mechanizm etykietowania dostosowany do bieżącej zmienności. Etykieta docelowa jest określana na podstawie przyszłego horyzontu obejmującego 5 dni. Próbką otrzymuje klasę pozytywną, czyli sygnał zakupu, jeżeli zmiana procentowa przekracza dynamiczny próg zdefiniowany przez kroczące odchylenie standardowe dziennych zwrotów przeskalowane do przyjętego horyzontu czasowego:

$$Próg = 1.5 \times \sigma_{dziennie} \times \sqrt{5}$$

Jeśli przyszły zwrot przekracza ten próg (przy zachowaniu dolnego limitu na poziomie 0,15%), obraz otrzymuje etykietę 1; w przeciwnym razie przypisywana jest etykieta 0, oznaczająca brak zakupu.

## 2.3 Architektura sieci ResNet-18

Zamiast budować sieć od podstaw, wykorzystano sprawdzoną architekturę ResNet-18, której połączenia skrótowe świetnie radzą sobie z wyciąganiem cech i zapobiegają problemom z uczeniem głębokich warstw.

Wprowadzono w niej dwie istotne modyfikacje. Po pierwsze, usunięto domyślną warstwę wyjściową rozpoznającą tysiące klas z bazy ImageNet na rzecz prostego klasyfikatora binarnego (0 lub 1). Po drugie, ponieważ czarne tło wykresów finansowych stwarza ryzyko, że sieć nauczy się po prostu ignorować puste piksele, wstrzyknięto warstwę Dropout o wartości 0.4. Zmusza to model do szukania wniosków za pomocą różnych ścieżek i uodparnia go na nowe dane.

## 2.4 Strategia trenowania i optymalizacja

Podejście do uczenia maszynowego w finansach wymaga dużej ostrożności, ponieważ łatwo o błędy prowadzące do zawyżonych wyników w testach i strat w rzeczywistości. Zastosowano następujące środki ostrożności:

- **Chronologiczny podział zbioru:** Rynki ciągle się zmieniają. Zamiast mieszać dane losowo, co powoduje wyciek informacji z przyszłości do zbioru treningowego, rygorystycznie oddzielono starsze dane (treningowe) od nowszych (walidacyjnych). To jedyny sposób, by sprawdzić model w realistycznych warunkach.
- **Radzenie sobie z większością:** Rynek Forex przez większość czasu po prostu konsoliduje. Bez dodatkowych poprawek sieć uznałaby, że najlepiej zawsze odpowiadać "0", bo i tak w większości przypadków będzie miała rację. Wymuszono więc równomierne losowanie (WeightedRandomSampler), by model równie często widział sygnały wybiecia, co braku akcji.
- **Inwestorska funkcja straty:** Zwykła sieć neuronowa traktuje każdy błąd tak samo. W tradingu liczy się jednak umiejętne "wstrzelenie się" w największe ruchy. Zmodyfikowano funkcję straty tak, by karała model mocniej, gdy ten pominie silne wybiecie, lub pomyli się tuż przed dużym krachem. Uczy to model priorytetyzacji – drobne pomyłki są ignorowane, o ile algorytm skutecznie łapie najmocniejsze, zyskowe trendy.

## 2.5 Szczegółowy proces przetwarzania i analizy danych

Proces przygotowania danych to ciąg kroków, które zamieniają zwykłe szeregi czasowe w obrazy gotowe do analizy przez sieć ResNet-18. Składa się on z sześciu etapów:

1. **Pobieranie i czyszczenie danych:** Za pomocą biblioteki `yfinance` ściągnięto historyczne, codzienne notowania dla dziesięciu głównych par walutowych (m.in. EUR/USD, GBP/USD, USD/JPY) z ostatnich 10 lat. Dane oczyszczono

z pustych wierszy (NaN) i ujednolicono do klasycznego formatu OHLCV (Open, High, Low, Close, Volume).

2. **Przygotowanie wskaźników:** Dane podzielono na przesuwające się okna o szerokości 20 dni. Dla każdego okna policzono popularne wskaźniki analizy technicznej: 10- i 20-dniową prostą średnią kroczącą.
3. **Dynamiczne etykietowanie:** Szytywne progi zysku źle sprawdzają się na giełdzie, dlatego algorytm dopasowuje się do bieżącej zmienności. Dla każdego 20-dniowego okna wyliczono odchylenie standardowe dziennych stóp zwrotu ( $\sigma$ ), a następnie skalowano je do 5-dniowego horyzontu (mnożąc przez  $\sqrt{5}$ ). Jeśli wzrost w ciągu kolejnych 5 dni przekraczał 1.5-krotność tego wskaźnika (przy ustalonym minimum na poziomie 0.15%), próbka dostawała etykietę "1" (sygnał zakupu). Pozostałe, słabsze ruchy oznaczano jako "0" (brak akcji).
4. **Generowanie obrazów:** Korzystając z biblioteki mplfinance, każdą 20-dniową sekwencję narysowano jako wykres świecowy z nałożonymi liniami SMA. Wykresy wygenerowano na czarnym tle, bez osi i napisów, w rozdzielczości  $224 \times 224$  piksele. Kluczowa była tutaj lokalna normalizacja (Min-Max). Ceny z każdego okna sprowadzono do przedziału  $[0, 1]$ . Dzięki temu sieć uczy się samych kształtów i relacji między świecami, a nie konkretnych historycznych poziomów cen.
5. **Podział i balansowanie danych:** Ponieważ okna czasowe przesuwają się za ledwie o jeden dzień, w dużej mierze się pokrywają. Aby uniknąć błędów, przez który algorytm zaglądałby w przyszłość, zrezygnowano z losowania. Dane podzielono ściśle chronologicznie: najstarsze 80% to zbiór treningowy, a najnowsze 20% to walidacyjny. Dodatkowo zastosowano mechanizm WeightedRandomSampler, który dba o to, by w trakcie nauki sieć widziała sygnały rzadkich wybić równie często, co rynkową konsolidację (klasę 0).
6. **Inwestorska funkcja straty:** Zamiast klasycznej funkcji błędów (Cross-Entropy Loss -  $L_{CE}$ ), zastosowano modyfikację opartą na wielkości ruchu:  $Strata = L_{CE} \times (1.0 + 5.0 \times |\Delta P|)$ . Błąd pomnożono przez faktyczną skalę ruchu cenowego ( $|\Delta P|$ ). Dzięki temu sieć jest mocniej karana za przegapienie dużych, zyskowych trendów, co naturalnie skłania ją do ignorowania rynkowego szumu i poszukiwania skoków.

## 2.6 Refleksje i wnioski z eksperymentów

Ostateczny kształt modelu to wynik wielu prób i błędów, które dobrze pokazują, z jakimi pułapkami wiąże się uczenie maszynowe na danych giełdowych:

1. **Wyciek danych (Look-Ahead Bias):** Na wczesnym etapie testowano standardowy, losowy podział zbioru (Random Split). Skutkowało to nierealistycznie wysokim wynikiem F1-Score na poziomie bliskim 95%. Okazało się, że przez nakładające się na siebie okna kroczące, sieć widziała w zbiorze treningowym dane, z których chwilę później była odpytwana w walidacji.

Algorytm nie mógł się pomylić, bo znał oczekiwany wynik. Dopiero cięcie chronologiczne (80/20) zweryfikowało faktyczne możliwości modelu.

2. **Złudzenie klasy większościowej:** Rynek przez większość czasu jest relatywnie stabilny, a duże wybicia stanowią rzadkość w stosunku do powolnych, długoterminowych trendów. Bez balansowania batchy model szybko orientował się, że najłatwiejszą strategią jest ciągle odpowiadanie "0" (brak akcji). W teorii dawało to wysoką ogólną celność, ale w praktyce inwestycyjnej było bezużyteczne. Użycie WeightedRandomSampler zmusiło się do realnego poszukiwania wzorców wybicia.
3. **Sens ważenia wielkości ruchu:** Tradycyjna funkcja straty traktowała każdą pomyłkę algorytmu tak samo. Dopiero wprowadzona modyfikacja  $(1.0 + 5.0 \times |\Delta P|)$  sprawiła, że model zaczął myśleć z perspektywy zysku i ryzyka. Przesłał skupiać się na wyłapywaniu gwałtownych, lecz niewielkich skoków (które są tylko szumem bez potencjału inwestycyjnego), a zaczął premiować te trendy, w których można było faktycznie zarobić.

### 3 Wyniki i ewaluacja

Ewaluację przeprowadzono na chronologicznie wydzielonym zbiorze walidacyjnym (20% najnowszych danych). Wyniki z perspektywy stricte analitycznej mogą wydawać się zbyt niskie by były warte uwagi (Tabela 1 i 2), jednak nabierają sensu w ujęciu biznesowym.

Model osiągnął bardzo wysoką dokładność ogólną (Accuracy: 88.06%). W praktyce oznacza to jednak, że stał się on niezwykle ostrożny i filtruje ogromną ilość sytuacji rynkowych jako zbyt ryzykowne (generując ponad 4400 prawidłowych odpowiedzi "Brak akcji"). Skutkiem ubocznym tak restrykcyjnego podejścia jest niska pełność (Recall: 13.17%) – model omija sporo zyskowych okazji, chcąc inwestować bez ryzyka, gdyż matematycznie jest to najlepsza opcja. Odbija się to mocno na syntetycznej metryce F1-Score, która wynosi zaledwie 0.1457.

Z punktu widzenia inwestora pominięcie okazji nie generuje jednak strat na rachunku. Najważniejsza pozostaje Precyzja (Precision: 16.32%). Oznacza ona, że mniej więcej jedna na sześć transakcji sugerowanych przez model trafia w silne wybicie cenowe. Przy zastosowaniu asymetrycznego systemu zarządzania kapitałem (gdzie szybko ucinamy małe straty z błędnych sygnałów, a pozwalamy rosnąć zyskom z dobrych wejść), taka skuteczność kierunkowa stanowi już wystarczającą bazę do osiągnięcia dodatniego bilansu inwestycji.

**Table 1.** Metryki wydajnościowe modelu ResNet-18 na zbiorze walidacyjnym.

| Metryka                        | Wartość (%) / Wynik |
|--------------------------------|---------------------|
| Accuracy (Dokładność ogólna)   | 88.06%              |
| Precision (Precyzja - Klasa 1) | 16.32%              |
| Recall (Pełność - Klasa 1)     | 13.17%              |
| <b>F1-Score (Klasa 1)</b>      | <b>0.1457</b>       |

**Table 2.** Macierz błędów dla prognoz binarnej klasyfikacji rynkowej.

|                        | Predykcja: Klasa 0<br>(No-Buy) | Predykcja: Klasa 1<br>(Buy) |
|------------------------|--------------------------------|-----------------------------|
| Rzeczywista<br>Klasa 0 | 4460                           | 241                         |
| Rzeczywista<br>Klasa 1 | 310                            | 47                          |

## 4 Wnioski

Badania dowodzą, że podejście bazujące na wizji komputerowej przy użyciu architektury ResNet-18 ma spory potencjał w analizie rynków. Przeprowadzając historyczne wykresy świecowe przez proces uczenia konwolucyjnego z odpowiednio zdefiniowaną, inwestorską funkcją straty, udało się uzyskać algorytm zdolny do autonomicznego wyodrębniania użytecznych cech z rynkowego szumu.

Rozwiązanie to ma jednak swoje ograniczenia. Model analizujący wyłącznie same grafiki cenowe ma dość duże i poważne ograniczenie. Wykres giełdowy to zaledwie jedna warstwa informacyjna. Sieć oparta na grafice nie jest w stanie zareagować na fundamenty gospodarcze, wpływające dane makroekonomiczne czy nastroje społeczne. Nie przewidzi nagłego spadku na rynkach wywołanego czarnym łabędziem, wybuchem konfliktu zbrojnego czy globalnym kryzysem, gdyż jest to unikatowe wydarzenie, matematycznie nieprzewidywalne.

Mimo tych ograniczeń, zaprezentowane podejście udowadnia, że komponent wizyjny może być wartościowy. Fakt, że korzystając wyłącznie z samego obrazu uzyskujemy mierzalną, użyteczną precyzję rynkową, stwarza świetne perspektywy na przyszłość. Rozwiązanie to może posłużyć jako fundament lub dedykowany moduł w rozbudowanych systemach hybrydowych, gdzie analiza wzorców geometrycznych za pomocą sieci CNN współpracuje równolegle z analizą sentymentu rynkowego (NLP) czy bieżącym monitorowaniem arkusza zleceń.

## Literatura

1. Paszke, A., et al.: PyTorch: An Imperative Style, High-Performance Deep Learning Library. *Advances in Neural Information Processing Systems* 32, 8024–8035 (2019) [Rdzeń obliczeniowy wykorzystany do budowy grafów i propagacji wstecznej]
2. Marcel, S., Rodriguez, Y.: Torchvision the Machine-Vision Package of Torch. W: *Proceedings of the 18th ACM International Conference on Multimedia*, s. 1485–1488 (2010) [Specjalistyczny moduł wizyjny wykorzystany do implementacji wag transfer learningu (ResNet18\_Weights) oraz transformacji obrazów]
3. Mplfinance developers: mplfinance: Financial Markets Data Visualization using Matplotlib. GitHub Repository (2023). Dostępne online: <https://github.com/matplotlib/mplfinance> [Wąskodziejowa biblioteka finansowa użyta do geometrycznego renderowania świec japońskich i wskaźników]
4. MetaQuotes Ltd.: MetaTrader 5 Python API. Algorithmic Trading and Quantitative Analysis (2023). Dostępne online: [https://www.mql5.com/en/docs/integration/python\\_meta-trader5](https://www.mql5.com/en/docs/integration/python_meta-trader5) [Specjalistyczne API wykorzystane do bezpośredniej integracji modelu ze środowiskiem egzekucji zleceń giełdowych]
5. Aroussi, R., et al.: yfinance: Yahoo! Finance market data downloader. Python Package Index (2023). Dostępne online: <https://pypi.org/project/yfinance/> [Narzędzie wykorzystane do ominięcia barier API i masowego pobierania historycznych danych giełdowych]
6. McKinney, W.: Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*, 51–56 (2010) [Wykorzystana do manipulacji szeregami czasowymi oraz budowy okien kroczących algorytmu]
7. Harris, C. R., et al.: Array programming with NumPy. *Nature* 585, 357–362 (2020) [Użyta do niskopoziomowych obliczeń matematycznych, w tym dynamicznego etykietowania zmienności]
8. Hunter, J. D.: Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95 (2007) [Silnik renderujący wykorzystany do ewaluacji modelu i generowania krzywych uczenia]